

Data Selection for LLMs

Paper Presentation

Joel Borja, Changxin Wan, Xingran Huang

EE 269, UC Riverside

May 21, 2026

Presentation Roadmap

1. Foundations & Motivation

Joel Borja

Motivation, taxonomy, and the LLM data pipeline

2. Methods & Techniques

Changxin Wan

Filtering, deduplication, scoring, and sampling

3. Applications & Future

Xingran Huang

Applications, benchmarks, results, and future directions

20 min presentation + 5 min Q&A per section — 5-minute break after Section 2

Part 1

Foundations & Motivation

Data Selection for LLMs

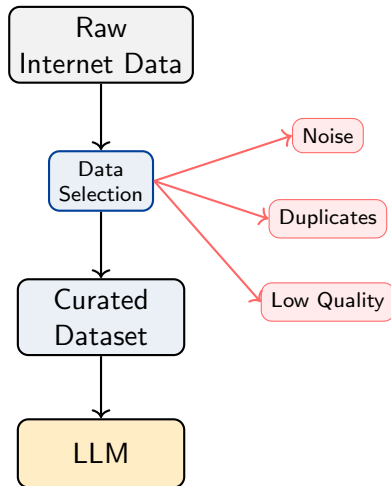
Why Data Matters for LLMs

Problems with Raw Internet Data

- Noisy and unstructured text
- Inconsistent data quality
- Duplicate or near duplicate data

Why This Matters

- Increased training costs
- Lower data efficiency
- Worse model performance



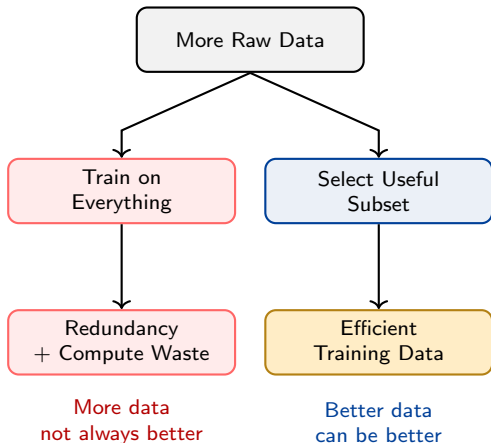
Why More Data Isn't Always Better

Problems with Scaling Raw Data

- Large datasets contain redundant information
- Duplicate text increases memorization risk
- Low-quality web data wastes compute
- More data does not always improve performance

Key Idea

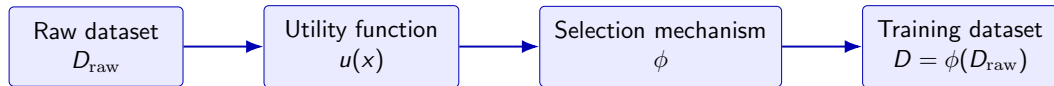
- Quality and diversity can matter more than size alone
- Data selection helps find the useful subset



Methods: What Does Data Selection Actually Do?

Core abstraction: Data selection transforms raw candidate data into a training dataset that better matches a target objective.

Formulation: $\arg \max_{\mathcal{D}} f_{obj}(\mathcal{M}), \quad \mathcal{D} = \phi(\mathcal{D}_{raw})$



Utility examples

- Language probability
- Quality score
- Domain similarity
- Duplication/toxicity score

Selection actions

- Keep or remove
- Edit or trim samples
- Upsample or downsample
- Reorder or select examples

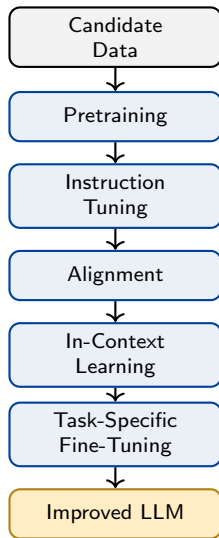
Training Stages Where Data Selection Applies

Data Selection Across Training

- Language model training occurs in multiple stages
- Each stage serves a different purpose
- Data selection can improve quality, efficiency, or task fit

Five Stages Identified in the Survey

- Pretraining
- Instruction tuning
- Alignment
- In-context learning
- Task-specific fine-tuning



Method Taxonomy: From General Filters to Concrete Algorithms

1. Pretraining

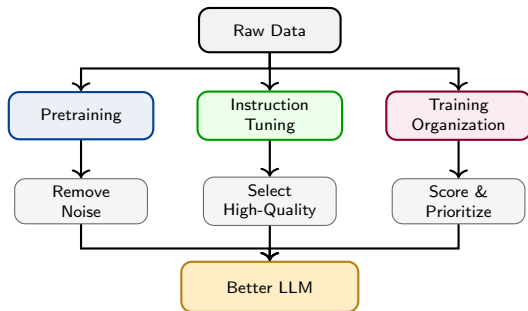
- Remove noisy web data
- Match target domains
- Example: **DSIR**

2. Instruction tuning

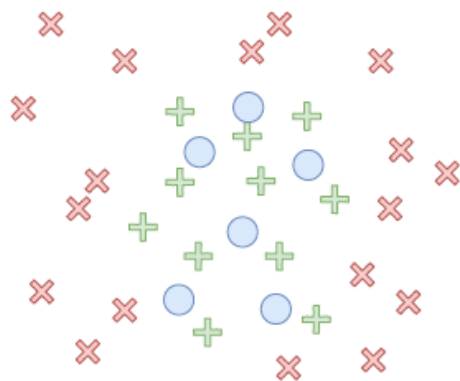
- Select high-quality instruction pairs
- Preserve diversity
- Examples: **IFD**, **DEITA**

3. Training organization

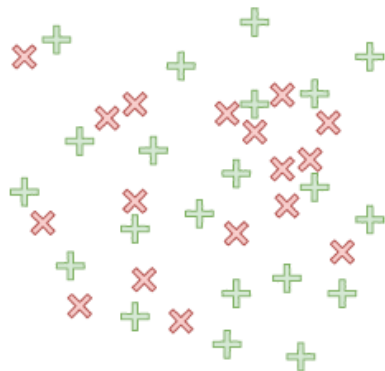
- Score and prioritize data
- Focus on useful samples
- Example: **DELT**



Distribution Matching vs Diversification



Distribution Matching



Distribution Diversification

Distribution matching selects target-relevant data, while diversification preserves coverage and reduces redundancy.

Diversification and Deduplication

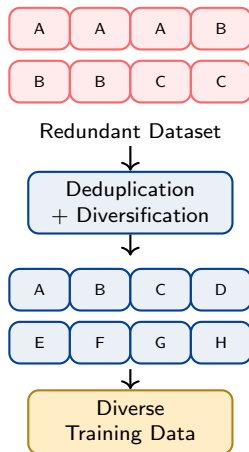
Why Redundancy Matters

- Web datasets often contain repeated content
- Duplicate data can increase memorization risk
- Redundant samples waste training compute

Why Diversification Helps

- Improves information coverage
- Encourages broader representations
- Increases data efficiency

Key idea: more unique information can be more valuable than more total tokens.



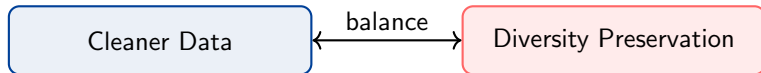
Challenges and Tradeoffs

Potential Benefits

- Cleaner and more efficient datasets
- Reduced redundancy and compute cost
- Better alignment with target tasks
- Improved training efficiency

Potential Risks

- Filtering can introduce bias
- Minority dialects may be removed
- “High-quality” is often subjective
- Aggressive filtering can reduce diversity



Effective data selection requires balancing quality, diversity, and representation.

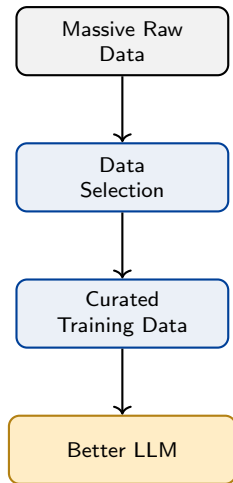
Key Takeaways

Main Insights

- Data quality matters as much as dataset size
- Data selection shapes model behavior and efficiency
- Different training stages require different selection strategies
- Distribution matching and diversification serve different goals
- Effective filtering involves important tradeoffs

Final Thought

Better data can be more valuable than simply more data.



Questions?

Part 2

Methods & Techniques

Data Selection for LLMs

Pretraining Pipeline: From Cheap Filters to Expensive Selection

Definition of Pretraining: To train a general purpose model with basic knowledge for all domains, which requires training on massive quantities of text (trillions of tokens)



Why this order?

- Raw corpora are huge; the obvious noise should be removed by cheap filters for efficiency.
- Later methods can be more targeted.

Important caveat

- This is a common pattern, not a universal law.
- Domains may change the pipeline.
- Thresholds depend on token budget.

Language and Heuristic Filters: Fast Signals for Obvious Noise

Language Filters: To filter the raw data into a specific type of language (English, Code),e.g., FastText (Small model with N n -gram)

Heuristic Filters: To filter the noise data (too long or short),e.g., C4 (remove the lines that have fewer than 5 words)

Heuristic family	Utility signal	Example action
Item count	characters, words, lines, sentences	remove too-short documents
Repetition count	repeated characters, words, n -grams, lines	remove boilerplate loops
Existence	word, phrase, or punctuation pattern appears	remove login prompts or blocked phrases
Ratio	alphabetic, numeric, uppercase, or symbol ratio	remove malformed text
Statistics	mean line length or line-length variance	flag abnormal web/code files

Key limitation

Heuristics are not truth detectors. They encode assumptions to the evaluation distribution and can over-filter useful legal, identity-related, or multilingual data.

Data Quality Filtering: Selecting Human-like, Edited Text

Goal: keep data that resembles a desirable reference distribution, often text believed to be human-written and edited.

Reference corpora

- Wikipedia
- books
- patents
- peer-reviewed articles
- curated web text

Classifier-based utility

- Positive: reference data
- Negative: raw web data
- Score: probability of being reference-like
- Often uses lightweight n-gram / fastText models

Perplexity-based utility

- Train an LM on reference text
- Score candidate documents
- Lower perplexity means closer to reference distribution

Important caveat

“High quality” is not universal. A filter trained on mainstream edited corpora can over-filter minority dialects, low-resource languages, or useful domain text.

Algorithm Detail: DSIR Matches Raw Data to a Target Distribution

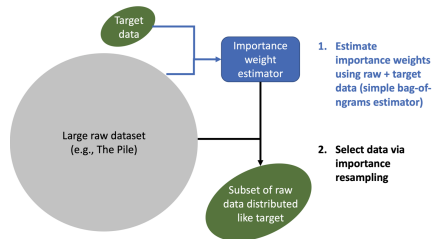
Problem setup: given raw data from $q(x)$ and a small target set from $p(x)$, select $k \ll N$ raw examples whose feature distribution looks like the target.

DSIR pipeline

- Featurize text: $z = h(x)$
- Use hashed unigrams + bigrams for speed
- Estimate $\hat{p}_{\text{feat}}$ from target data
- Estimate $\hat{q}_{\text{feat}}$ from raw data
- Compute importance weights:

$$w_i = \frac{\hat{p}_{\text{feat}}(z_i)}{\hat{q}_{\text{feat}}(z_i)}$$

- Sample k examples without replacement by w_i



Why this improves over simple filters

- It targets the full feature distribution, not only top-scoring examples.
- KL reduction measures whether selected data moves closer to target.

Deduplication: Remove Redundancy Without Killing Useful Memory

Deduplication: to remove duplicates so that the training distribution provides greater coverage with less redundancy (Levenshtein distance)

Common deduplication utilities

- **URL deduplication:** cheap first pass for web data
- **Exact hash matching:** detect identical documents
- **Exact substring search:** detect long copied spans
- **MinHash / LSH:** detect near duplicates
- **Model-based semantic deduplication:** detect meaning-level redundancy

Why it matters

- reduces training waste
- increases distribution coverage
- lowers memorization risk
- helps prevent test contamination

Tradeoff

Some memorization is useful for facts, APIs, and syntax. The goal is to reduce harmful memorization, not erase all repetition.

Toxicity, Explicit Content, and PII Filters

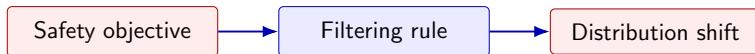
Goal: for models trained on such data to follow similarly unbiased behavior

Common filtering mechanisms

- URL or domain blocklists
- lexicon / keyword matching
- toxicity or NSFW classifiers
- models trained on undesirable content
- regex or classifiers for PII and secrets

No free lunch

- Toxicity filters can reduce toxic generations.
- But they may also remove useful examples for detecting toxic content.
- They may disproportionately remove text from certain communities or topics.



Multilingual Selection: Same Pipeline, Different Parameters

What can be reused?

- language identification
- deduplication
- technical-character heuristics
- data mixing across languages

What must be adapted?

- length thresholds by language
- script and encoding filters
- language-ID confidence thresholds
- native-speaker quality checks for low-resource languages

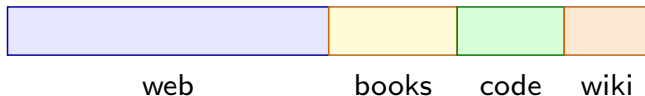
Example intuition

Chinese can carry more information per character than English, so an English minimum-length threshold may incorrectly remove useful Chinese text.

Data Mixing: Sampling Weights Are Also Data Selection

When a corpus has multiple domains, data selection also decides how often each domain appears during training.

α : domain mixture weights



How weights are chosen

- natural data proportions
- manually chosen weights
- small-model empirical search
- principled updates: DoReMi, DoGE
- online updates during training

Core tradeoff

Upweighting one source necessarily downweights others under a fixed training budget. Good weights may not transfer across tokenizer, model scale, or training length.

From Data Selection to Data Efficacy

Earlier pretraining methods mostly ask: which examples should enter the training set?

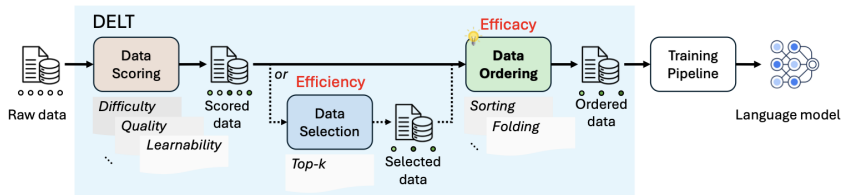
Question: How can we sample the training dataset efficiently?

Data efficiency

- Goal: use fewer / better samples.
- Methods: filtering, sampling, deduplication, DSIR.
- Limitation: retained samples are usually treated equally.

Data efficacy

- Goal: make the same data work better.
- Key question: In what order should the model see samples?
- DELT adds data organization to data selection.



DELT: Scoring, Optional Selection, and Folding Ordering

1. Score each sample

$$\gamma = f(D) = [\gamma_1, \dots, \gamma_{|D|}]^\top$$

$$L(x_n) = \sum_{t=1}^{T-1} \frac{\|\nabla \ell(x_n, \theta_t)\|}{\|\nabla \ell(x_n, \theta_{t+1})\|}$$

$$Q(x_n) = \sum_{t=1}^{T-1} \frac{\lambda_{t+1}^\top \nabla \ell(x_n, \theta_t)}{\|\lambda_{t+1}\| \|\nabla \ell(x_n, \theta_t)\|}$$

LQS combines learnability L and gradient-alignment quality Q .

2. Optionally select top data

$$K = \lfloor r|D| \rfloor, \quad D_{\text{sub}} = \{x_k : \text{rank}(\gamma_k) > |D| - K\}$$

3. Reorder data, not just shuffle

$$D' = f_o(D; \gamma) = [x_{\pi(\gamma)_1}, \dots, x_{\pi(\gamma)_{|D|}}]$$

Sorting gives a curriculum, but can create forgetting and distribution bias.

Folding Ordering

$$\pi_{\text{fold}}(\gamma; L) = \bigcup_{\ell=0}^{L-1} \langle \pi_{\text{sort}}(\gamma)_i : i \equiv \ell \pmod{L} \rangle$$

Sort by score, then interleave score ranges. This keeps curriculum structure while reducing long blocks of similar data.

Takeaway

DELT is about **how to use selected data**: score it, optionally filter it, then order it for training.

Instruction Tuning: Four Families of Selection Algorithms

Family	Core idea	Representative methods
Indicator systems	Combine explicit metrics such as length, perplexity, reward score, diversity, or embedding features.	INSTRUCTMINING, DQ
Trainable LLM selector	Fine-tune or adapt an LLM to score instruction data.	IFD, Instruction Backtranslation, ActiveIT
Powerful LLM judge	Prompt GPT-4 / ChatGPT to rate quality, complexity, or tags.	AlpaGasus, INSTAG, LIFT, DEITA
Small-model / coreset	Use smaller selectors or coverage objectives to reduce selection cost.	MoDS, coreset-based selection

Why this differs from pretraining

Instruction tuning data is much smaller, so the key objective shifts from cheap large-scale filtering to selecting examples with high quality, high diversity, and strong instruction-following signal.

Algorithm Detail: IFD, AlpaGaus, INSTAG, and DEITA

IFD: instruction-following difficulty

- Score how much the instruction helps generate the answer.
- Compare answer likelihood with and without instruction:

$$r_{\theta}(Q, A) = \frac{s_{\theta}(A|Q)}{s_{\theta}(A)}$$

- Keep informative examples above a threshold.

AlpaGaus

- Ask ChatGPT to rate each instruction-input-output tuple.
- Filter tuples below a quality threshold.
- Simple but depends on judge quality and cost.

INSTAG

- Use ChatGPT to tag semantic intent and task attributes.
- Prefer samples with more / rarer tags.
- Selection objective: complexity-first diversity.

DEITA

- Score instruction complexity c and response quality q .
- Rank by a combined score such as $c \times q$.
- Enforce diversity by nearest-neighbor distance threshold.

Post-training Methods: Selection Depends on the Feedback Signal

SFT

- Data: instruction $\rightarrow$ response
- Select prompts that cover target tasks
- Fill weak benchmark areas with new or synthetic data

DPO / preference tuning

- Data: prompt + chosen + rejected
- Keep high-confidence preference pairs
- Judge can be human, GPT-4, or reward model

RLVR

- Data: verifiable tasks such as math/code
- Reward is correctness from a scoring function
- Less subjective than open-ended preference labels

Practical tradeoff

Synthetic judges can be low-noise but biased; human labels are less biased but noisier and more expensive.

Practical Recipe for Choosing a Method

1. **Choose the training stage.** Pretraining, instruction tuning, preference tuning, or continued domain training?
2. **Choose the objective.** Model performance, data efficiency, evaluation integrity, safety, or selection efficiency?
3. **Pick the scoring signal.** Heuristics, DSIR importance weights, IFD score, LLM judge score, LQS, or reward score.
4. **Pick the selection action.** Filter, importance-resample, diversify, reweight, rank, or reorder.
5. **Audit the side effects.** False positives, data bias, benchmark leakage, memorization, distribution shift.

Takeaway: data selection is training-distribution design.

References Used for the Methods Section

- Albalak et al. *A Survey on Data Selection for Language Models*. TMLR, 2024.
<https://arxiv.org/pdf/2402.16827>
- Xie et al. *Data Selection for Language Models via Importance Resampling*. 2023.
<https://arxiv.org/abs/2302.03169>
- Zhang et al. *A Survey on Data Selection for LLM Instruction Tuning*. 2024.
<https://arxiv.org/pdf/2402.05123>
- Dai et al. *Data Efficacy for Language Model Training*. 2025.
<https://arxiv.org/pdf/2506.21545>
- Sanchez Hernandez. *Notes on “Opening the LLM pipeline: data, adaptation, evaluation”*. NeurIPS 2024 tutorial notes, 2025.
<https://manuelsh.github.io/blog/2025/NIPS-building-llm-workshop/>
- Data selection survey code and bibliography.
<https://github.com/alon-albalak/data-selection-survey>

Methods section scope: survey framework + concrete algorithms + post-training data selection.

Break

5-Minute Intermission

Please return shortly

Part 3 Roadmap: Applications, Results, Future

Applications: Real systems using data selection across three lifecycle stages.

Results: DSIR on GLUE; instruction-tuning gains; DELT data efficacy gains.

Future: Evaluation + open-data; selection cost; multilingual + multimodal; compute rebalancing.

Synthesizing 4 papers + 1 survey GitHub + 1 NeurIPS tutorial blog.

Applications - Deployment Across the LLM Lifecycle

Stage	Selection objective	Real systems
Pretraining	Distribution matching to formal text; heuristic + classifier filtering	GPT-3, PaLM (heuristic classifier); Refined-Web / Falcon; OLMo / Dolma
Continued pre-training	In-domain importance resampling	DAPT, DSIR-style domain adaptation
Instruction tuning	Quality / complexity / diversity filtering	LIMA (1k examples); AlpacaGPT; Tulu 2
Preference / alignment	High-confidence pairs; verifiable rewards	Llama 3 (DPO); Tulu 3 (RLVR)

Takeaway

Every widely deployed open LLM applies data selection at multiple lifecycle stages.

Sources: Xie et al. 2023 (DSIR); Zhang et al. JAIR 2025 (LIMA, AlpacaGPT); Lambert et al. 2024 (Tulu 3 / RLVR); blog notes from NeurIPS 2024 tutorial.

Applications Case Study - DSIR on The Pile (1.6B examples)

Two deployments of one algorithm, differing only in the target distribution.

Use case 1: Domain-specific continued pretraining

- 8 downstream targets across 4 domains: CS (ACL-ARC, SciERC); Biomed (ChemProt, RCT); News (HyperPartisan, AGNews); Reviews (Helpfulness, IMDB)
- Per-target resampling $\rightarrow$ per-domain RoBERTa checkpoint

Use case 2: General-domain pretraining from scratch

- Target: Wikipedia + books (proxy for formal text)
- 51.2M examples, 4 epochs, BERT-base
- Evaluated on GLUE dev (5-seed averaging)

Significance

The hashed-n-gram importance estimator generalizes from narrow domain adaptation to broad pretraining purely by swapping the reference set $\mathcal{T}$.

Results: DSIR on GLUE - Distribution Matching Beats Heuristics

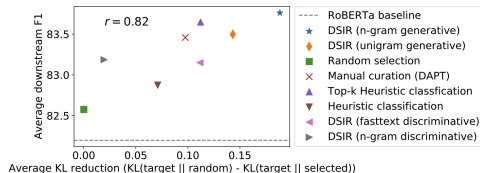


Figure 3: Plot of average KL reduction on the n-gram feature space, defined as how much the selected dataset reduces KL divergence to the target distribution over just random sampling from The Pile, against average downstream F1 score over the 8 continued pretraining datasets in Table 1. There is a strong correlation between KL reduction and downstream performance (Pearson $r = 0.82$).

Source: Xie et al., *Data Selection for Language Models via Importance Resampling*, NeurIPS 2023, Figure 3

Figure description: KL reduction on hashed n-gram features (x-axis) vs. average downstream F1 (y-axis), across 8 selection methods.

Empirical observation:

- DSIR (n-gram generative) lies at the top-right — highest KL reduction *and* highest F1.
- **Pearson $r = 0.82$.**

Implication: A purely statistical closeness metric (KL on n-grams) predicts downstream model performance. Selection quality is measurable *without* training a model.

Results: DSIR on GLUE - Raw Numbers

General-domain LM, BERT-base trained on 51.2M examples from The Pile, evaluated on GLUE dev (5 seeds).

Method	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	STS-B	Avg
Random selection	82.63	86.90	89.57	67.37	90.05	87.40	49.41	88.63	80.25
Heuristic (GPT-3)	82.69	85.95	89.77	68.59	88.94	86.03	48.17	88.62	79.85
Top- <i>k</i> heuristic	83.34	88.62	89.89	70.04	91.15	86.37	53.02	89.30	81.47
DSIR	83.07	89.11	89.80	75.09	90.48	87.70	54.00	89.17	82.30
Top- <i>k</i> DSIR	83.39	88.63	89.94	72.49	91.01	86.18	49.90	89.52	81.38

Key observations:

- DSIR beats the GPT-3 heuristic by **+2.5%** on average (82.30 vs. 79.85).
- Largest gains on small / hard tasks: RTE **+6.5%**, CoLA **+5.8%**.
- Importance resampling beats top-*k* deterministic selection by ~1%.

Throughput

DSIR processes **100M documents from The Pile in 4.5 hours** on a single 96-core CPU node.

Source: Xie et al., *Data Selection for Language Models via Importance Resampling*, NeurIPS 2023, Table 3.

Results: Instruction Tuning - Less Data, Higher Quality

Source: Zhang et al. JAIR 2025 survey, Table 4 (External Comparison). Models tuned on small selected subsets compared against external LLMs.

Method	Training subset	Base model	MT-Bench	AlpacaEval
GPT-4 (reference)	—	—	8.99	95.28
GPT-3.5-turbo (reference)	—	—	7.94	91.36
Alpaca-13B (full)	full Alpaca	Llama-13B	4.53	—
NUGGETS	7.5k of Alpaca	Llama-7B	5.34	—
Random sampling	mixture	Llama-13B	5.84	—
IFD	mixture	Llama-13B	5.91	—
TAGLM-13B-v1.0 (IN-STAG)	6k of mixture	Llama-13B	6.44	72.8
DEITA	10k of mixture	Llama2-13B	6.79	81.09

What it means

DEITA on 10k carefully selected examples reaches **6.79** on MT-Bench — a **50%** jump over the full-data Alpaca baseline (4.53). IFD additionally beats the full Alpaca with only **5%** of its data.

Results: DELT — Reordering the Same Data Lifts Performance

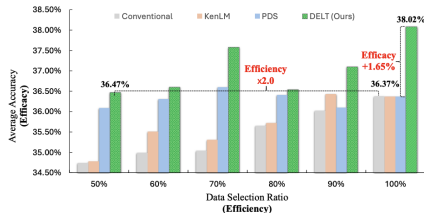


Figure 1: Average result across 8 benchmarks for different methods. High performance at the same selection ratio indicates high efficacy, while achieving similar performance with a smaller selection ratio demonstrates high efficiency. Our method excels in both efficacy and efficiency.

Source: Dai et al., *Data Efficacy for Language Model Training*, Microsoft Research 2025, Figure 1.

Figure description: 4 curves (Conventional / KenLM / PDS / DELT) on the same axes: avg accuracy across 8 benchmarks vs. data selection ratio.

Empirical observation:

- Same data (100%): DELT **38.02** vs. Conv. **36.37** $\Rightarrow$ **+1.65%**.
- Same accuracy (36.47): DELT needs **60%** data $\Rightarrow$ **2 $\times$** efficiency.

Implication: Data *efficacy* (better order) and data *efficiency* (less data) can be achieved together. Ordering is a new design axis.

Results: DELT — Gains Are Consistent Across Scale

Source: Dai et al. 2025, Table 1. OLMo evaluation suite average across 8 benchmarks (ARC-c, ARC-e, HS, LAMB, OBQA, PIQA, SciQ, Wino).

(a) Across model sizes (1B-token data)

Model	Conv.	Ours (DELT)	Δ
160M	36.37	38.08	+1.71
470M	36.85	38.33	+1.48
1B	37.77	39.17	+1.40

(b) Across data scale (160M model)

Data	Conv.	Ours	Δ
1B tok	36.37	38.08	+1.71
10B tok	40.24	41.62	+1.38
50B tok	42.20	43.03	+0.83

Domain post-training (Qwen1.5-1.8B, Table 3):

- Code: HumanEval 9.15 $\rightarrow$ **16.46**; MBPP 12.00 $\rightarrow$ **13.20**
- Math: MathQA 22.72 $\rightarrow$ **24.75**; GPQA-Diamond 27.17 $\rightarrow$ **28.94**

What it means

Gains hold from 160M to 1B parameters and from 1B to 50B tokens. The improvement narrows at larger scale but never disappears.

Source: Dai et al., *Data Efficacy for Language Model Training*, 2025, Tables 1 & 3.

Results: Cross-Paper Synthesis

1. **Quality dominates quantity at small scale.** LIMA (1k examples), IFD (5% of Alpaca) both beat full-data baselines.
2. **Distribution targeting beats ad-hoc thresholding.** DSIR matches expert curation (DAPT) within 0.3% on average.
3. **Ordering is a third design axis.** DELT achieves +1.65% with no change to data or model.
4. **Inner vs. external evaluation can disagree.** AlpaGasus excels in inner comparison but is unstable across external LLMs.

Methodological message

Selection quality is benchmark-conditioned. Reports based on a single benchmark or a single judge are insufficient.

Sources: Zhang et al. JAIR 2025 §1 (LIMA, IFD); Xie et al. 2023 §5 (DSIR vs DAPT); Dai et al. 2025 Fig. 1 (DELTA +1.65%); Zhang et al. JAIR 2025 §4.3 (AlpaGasus).

Future 2: Selection Cost and Scalability

Bottleneck today

- LLM-as-judge methods (DEITA, AlpaGasus) call GPT-4 per sample.
- Filtering hundreds of thousands of instructions is “*prohibitively expensive*” (JAIR §5).
- Datamodel-based selection has higher compute requirements than DSIR (Albalak survey §3.4).

Proposed research directions

- **Distillable selectors**: compact models emulating GPT-4 scoring.
- **Retrieval-augmented pruning**: cheap heuristic shortlist → LLM rerank on top- k .
- **Direct data metrics**: training-free utility (Albalak survey §11.1).

Open challenge

Most data utility functions are coupled to downstream model training, creating a chicken-and-egg loop between selection quality and evaluation cost.

Sources: Zhang et al. JAIR 2025, §5; Albalak et al. TMLR 2024, §3.4 & §11.1.

Future 3: Beyond English, Beyond Text

Multilingual & domain coverage

- Most selection frameworks designed for English + general tasks.
- Open: Chinese, Arabic, low-resource languages.
- Open: biomedical, legal, programming.
- Proposed: multilingual encoders (XLM-R) + modular pipelines with swappable domain priors.

Multimodal extension

- DELT (Appendix C): *“no evaluation in other modalities like image and audio.”*
- Direction: scale to hundred-billion-parameter models, terabyte-level datasets.
- Open: gradient-consistency scoring (LQS) on non-text data.

Why modularity matters

A single pipeline cannot serve medical, legal, multilingual, and code domains. Component-level swapping is the architectural answer.

Future 4: Rebalancing Compute Toward Data

A structural shift in how compute is allocated.

Classical allocation

- Compute dominated by model training.
- Data preprocessing is a one-time fixed cost.
- Selection is largely heuristic.

Emerging allocation

- More compute on data scoring, selection, and ordering.
- Methods built for low-data alignment now applied to pretraining scale.
- DELT formalizes ordering as a first-class axis.

Albalak et al. 2024, §11.3 (verbatim)

“It is now well known that training data plays a very significant role in the performance of a model, therefore creating an increased desirability to spend more compute on data processing and selection (and possibly away from model training).”

Conclusion: Synthesis of 6 Resources

- **Applications.** Data selection spans pretraining (DSIR, GPT-3, OLMo), instruction tuning (LIMA, AlpacaGasus, DEITA), and alignment (Llama 3 DPO, Tulu 3 RLVR).
- **Results.** The numbers are concrete:
 - DSIR: **+2.5%** GLUE over heuristic; 100M docs in 4.5h.
 - IFD: full-data Alpaca beaten with **5%** of the data.
 - DEITA: MT-Bench **4.53** → **6.79** with only 10k examples.
 - DELT: **+1.65%** from reordering alone; **2×** data efficiency.
- **Future.** Standardized evaluation, lower selection cost, multilingual / multimodal extension, and a structural compute rebalance toward data.

Data selection has matured from preprocessing into a first-class research area.

Questions?

References

1. Albalak et al. *A Survey on Data Selection for Language Models*. TMLR, 2024.
arxiv.org/abs/2402.16827
2. Xie et al. *Data Selection for Language Models via Importance Resampling (DSIR)*. NeurIPS, 2023. arxiv.org/abs/2302.03169
3. Zhang et al. *A Survey on Data Selection for LLM Instruction Tuning*. JAIR, 2025.
arxiv.org/abs/2402.05123
4. Dai et al. *Data Efficacy for Language Model Training (DELT)*. Microsoft Research, 2025.
arxiv.org/abs/2506.21545
5. Sánchez Hernández. *Opening the LLM Pipeline*. NeurIPS 2024 Tutorial Notes, 2025.
manuelsh.github.io/blog/2025/NIPS-building-llm-workshop
6. Albalak. Companion Repository: github.com/alon-albalak/data-selection-survey